IN-DATABASE FEATURE STORES

USING A LIGHTWEIGHT ANALYTICAL DATABASE (DUCKDB)

References & Results

01 What is a Feature Store?

A feature store **computes and stores information (features)** about an entity's recent history – and serves it for **two downstream consumers**:

1. Training machine learning models (offline)
2. Real-time applications for ML inference (online)

Temporal features (e.g. rolling aggregates over a window) are the main focus of Feature Stores

Simplified Explanation of Feature Stores*

Built-in scheduler computes features on a timed interval

↓

Features are materialised and stored alongside a version history

↙ ↘

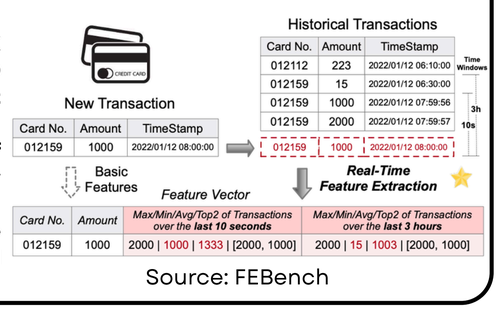
Training – needs the feature as it was known at the time of each labelled event.

Inference – needs the feature as of right now, in a short time.

**The exact design and implementation varies*

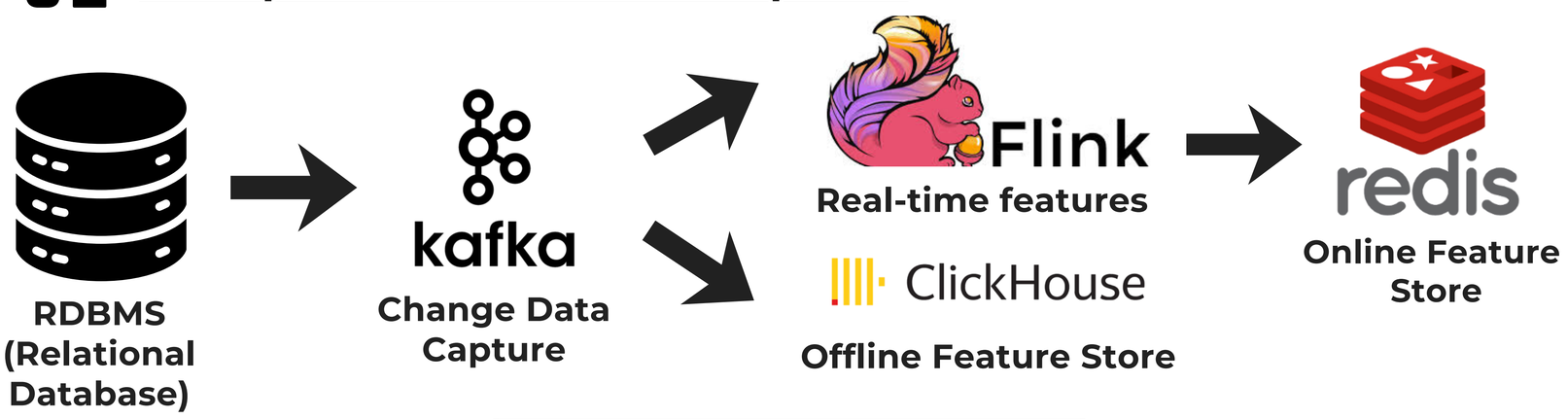
Worked Example

- A data scientist is tasked to detect fraudulent transactions.
- She discovers that the **MAX transaction in the last 10 seconds is an important feature** for her model
- She needs a **large dataset of this window features to train** her model (offline)
- She wants the **feature to be retrieved quickly for model inference** (online)



Source: FEBench

02 Enterprise Feature Stores Pipeline



However, there are some issues...

1. ***Training/serving skew*** — **2x** Implementation (Offline & Online); complexity for maintenance; chance of feature drift
2. ***Edge deployment*** — IoT gateways / embedded systems often **cannot host a distributed cloud stack.**
3. ***Resource constraints*** — **Small teams with limited budgets cannot maintain** a multi component distributed pipeline.
4. ***Network round-trips*** — **Low-latency local inference** needs features co-located with the data + model on the same node.

Research Question: Can a single RDBMS serve as a self-contained feature store?

03 Important Feature Operations

REFRESH (PIT Join)

Query

At timestamp *ts*

For every distinct entity, calculate the **windowed aggregate in the half-open interval [ts - window, ts)**

```
SELECT u.user_id,
COALESCE(SUM(o.amount), 0) AS
total_spend
FROM users u
LEFT JOIN orders o
  ON  o.user_id  = u.user_id
 AND  o.order_ts >= ts -
INTERVAL '7 day'
 AND  o.order_ts <  ts
GROUP BY u.user_id;
```

Point-in-time (PIT) Join SQL Query

Cost

N rows, N_w inside the window, E entities:

Scan + Hash Aggregate **$O(N_w)$** + Entity projection **$O(E)$** + Hash left join **$O(N_w + E)$**

SERVE (ASOF Join)

Query

For each (entity, request_time) pair in incoming requests

Match it with the **most recent feature snapshot at or before the request time**, NULL otherwise.

```
SELECT requests.*, f.total_spend
FROM requests
ASOF LEFT JOIN feature_store f
ON requests.user_id = f.user_id
AND requests.request_time >=
f.__feature_timestamp;
```

ASOF Join SQL Query

Cost

DuckDB's ASOF is a partitioned sort-merge join
m request entries, E entites, R versions:

Request Sort **$O(m \log m)$** + Feature Store sort **$O(ER \log(ER))$**

04 Why DuckDB?

DuckDB is an **in-process** (engine runs inside same memory address space and OS as host application) **OLAP** database

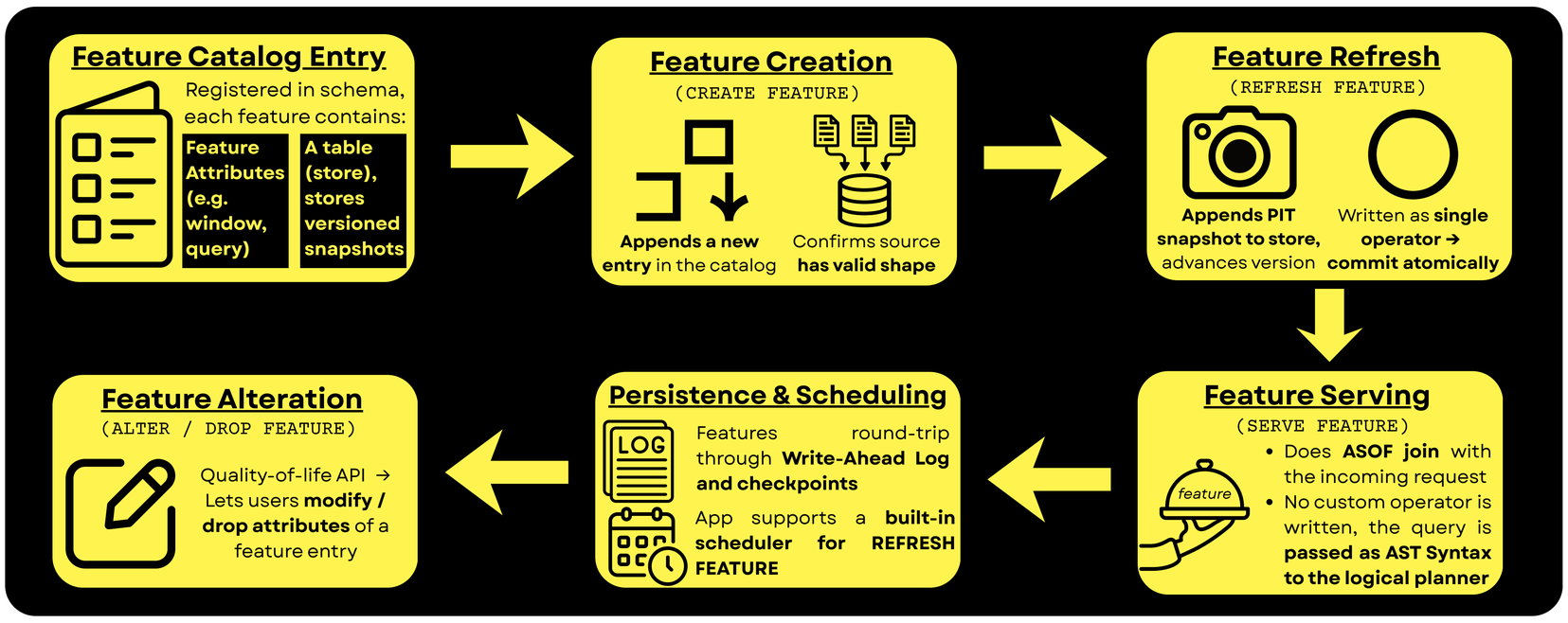
1. Has a **native ASOF join** implemented and optimized
2. **Vectorized execution parallelizes** feature operations
3. **Columnar Storage** lets range predicates filter faster
4. **MVCC** allows multiple operations to commit as a single transaction

Syntax for Feature Operations

```
CREATE FEATURE spend_7d
ENTITY users (user_id) -- the entity dimension
TIMESTAMP order_ts -- event-time column driving the window
WINDOW 7 DAYS -- lookback width
TTL 1 HOUR -- serving staleness bound
EVERY 5 MINUTES -- built-in scheduler
RETAIN 20 -- snapshots kept
AS (SELECT user_id, SUM(amount) FROM orders GROUP BY user_id);

REFRESH FEATURE spend_7d [AT '2024-01-04']; -- append a snapshot
SELECT * FROM FEATURE spend_7d AT VERSION 3; -- time travel
SERVE FEATURE spend_7d FOR live_txns ASOF label_time; -- ONLINE
ALTER FEATURE spend_7d SET TTL 30 MINUTES;
```

05 Design and Implementation



06 Optimizations

Utilizing DuckDB's Zonemap

DuckDB's columnar storage **stores 122,800 entries in one row-group**

Table
Row Group 1
Row Group 2
...

Each row-group has **zonemaps – cheap min/max statistics**

Can **skip row-groups** if zonemap proves matching predicate is impossible

↑ **Effectiveness**

REFRESH appends snapshots → **store table is clustered by feature_version and timestamp**

Eliding DISTINCT

Problem	Finding	Solution
`SELECT DISTINCT` in REFRESH → a **full hash aggregate** over the entity table	`PRIMARY KEY` If entity column is **PK**, the **keys are NON-NULL and unique**	`ADDITIONAL CHECK` If entity column is a **PRIMARY KEY**, we **skip DISTINCT**

GC zonemap row-group filter

Problem	Finding	Solution
Deletion of rows is entirely logical until next checkpoint	A **row evicted by an earlier refresh is present** → zonemap reports stale version	Additional zonemap predicate filter `feature version` ≥ `oldest live version`

↑ **REFRESH**

Bounding the store scan

Problem	Finding	Solution
Row-groups with min timestamp > request's max **can't be the solution, are read**	A store row **can only be an answer if its timestamp precedes request's** timestamp	**Push filter zonemap check**, where reachable range spans R' ≤ R versions

Version-table equi-join

Problem	Finding	Solution
Serve ASOF-joins the request **against the whole store**	Version resolution **needs only (version, timestamp) pairs**, at most R of them.	**Split** one large join **into two** 1. ASOF-join against (version, timestamp) 2. Equi-join on (entity keys, version)

↑ **SERVE**

07 Benchmark

Setup

Dataset: **Hits** from Clickbench

Parquet files	Total number of rows	Distinct Users
1	1,000,000	79,842
10	10,000,000	1,530,334
100	99,997,497	17,630,976

Specs / Environment: Intel Core Ultra 9 185H (11C/22T) · 15 GiB RAM · Ubuntu 22.04 / WSL2

Results are the **mean of 3 runs**, tested **with / without optimizations**

Methodology

Each scenario is sampled by a **seeded pairwise covering array** of varied configurations → **simulate randomized testing**

Worked Example

refresh_s100_multi_w1d_r5_prior5.benchmark

→ Refresh benchmark

Source size of 100 files — Multi aggregation

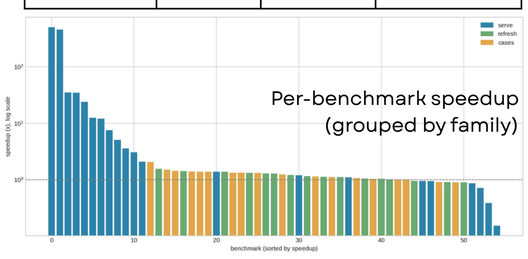
1 day window — 5 retained versions

5 prior refreshes

serve/ – 21 scenarios
refresh/ – 16 scenarios
cases/ – 6 curated multi-step scenarios

Results

Family	Baseline (s)	Optimized(s)	% Speedup
serve	168.235	1.164	99.3%
refresh	9.663	8.427	12.8%
cases	33.381	25.304	24.2%
total	211.279	34.895	83.5%



Per-benchmark speedup (grouped by family)

08 Discussions

Analysis

serve — **Largest speedup** due to the **new equi-join plan**

refresh / cases — Most gains come from **large file sizes** (GC optimizations)

regressions — 12 of 55 **Each under 0.16 s:** jitter / unable to amortize on small inputs

Follow up ideas / questions

Index — Future optimizations using database indexes?

1 Operator — Could we define a shared operator used by both REFRESH and SERVE?

Vectors — Could this feature store design extend to other temporal features (vector embeddings)?

Isaac Ong
Advised by Zeng Lingze